pytorch_learning_from_scratch

2026 年 3 月 24 日

1 torch 学习

1.1 torch 基础

1.1.1 定义 tensor

[13]:
```
import torch
a = torch.Tensor([[1,2,3],
                  [2,3,4]])
input_dim = 2
output_dim = 2
W = torch.randn(input_dim, output_dim)
b = torch.zeros(output_dim)
x = torch.Tensor([[2,3],
                  [4,5]])
y_true = x.T @ x
```

1.1.2 自动求导

[14]:
```python
W.requires_grad = True
b.requires_grad = True
y = x @ W + b
loss = torch.mean((y - y_true)**2)
loss.backward()
print(f"loss 对 W 的偏导为{W.grad}")
print(f"loss 对 b 的偏导为{b.grad}")

# 由于 PyTorch 中梯度会累加，所以必须清零，这样每次输出的都是真实梯度
# W.grad.zero_()
# b.grad.zero_()
```

```
loss 对 W 的偏导为 tensor([[ -69.4081,  -95.0080],
        [ -91.5912, -125.3297]])
loss 对 b 的偏导为 tensor([-22.1831, -30.3217])
```

1.1.3 with 语句学习

with 语句演示

[15]:
```python
class GPUSession:
    def __init__(self, memory_required):
        self.memory = memory_required
        print(f"初始化：准备申请 {self.memory}GB 显存...")

    def __enter__(self):
        print(">> __enter__ 被调用：成功锁定 GPU 显存，开始仿真！")
        return self  # 这个返回值会赋给 as 后面的变量

    def __exit__(self, exc_type, exc_value, traceback):
        print(">> __exit__ 被调用：释放 GPU 显存，清空缓存！")
        if exc_type is not None:
            print(f"   [警告] 实验中途发生了异常：{exc_value}")
        # 注意：这里没有所谓的 close() 方法，清理逻辑全在 __exit__ 里
        return True

# 使用我们自己写的类：
print("--- 实验开始 ---")
with GPUSession(16) as session:
```

```
    print("正在进行偏微分方程求解...")
    # 假设这里发生了一个计算错误
    raise ValueError("矩阵维度不匹配！ ")
print("--- 实验结束 ---")
```

```
--- 实验开始 ---
初始化：准备申请 16GB 显存…
>> __enter__ 被调用：成功锁定 GPU 显存，开始仿真！
正在进行偏微分方程求解…
>> __exit__ 被调用：释放 GPU 显存，清空缓存！
   [警告] 实验中途发生了异常：矩阵维度不匹配！
--- 实验结束 ---
```

with 语句实践

[16]:
```
import time
class Timer:
    def __init__(self,name):
        self.name = name
        self.time = None
    def __enter__(self):
        self.time1 = time.time()
        return self
    def __exit__(self,exc_type,exc_val,tb):
        self.time2 = time.time()
        print(f"{self.name}耗时{self.time2-self.time1}秒")
        if exc_type is not None:
            print(exc_val)
        return True
with Timer("test"):
    time.sleep(0.5)
    raise ValueError("数值错误")
```

```
test 耗时 0.5050759315490723 秒
数值错误
```

with 语句改进

[17]:
```python
import time
from contextlib import contextmanager

@contextmanager
def elegant_timer(name):
    # 这里相当于 __enter__ 的部分
    start_time = time.time()

    try:
        # yield 是关键！它把执行权暂时交给了 with 语句内部的代码块
        yield name

    finally:
        # 这里相当于 __exit__ 的部分（无论是否报错都会执行）
        end_time = time.time()
        print(f"[{name}] 耗时 {end_time - start_time} 秒")

# 使用方式完全一样：
with elegant_timer("[张量点乘测试]") as name:
    print(f"{name}开始")
    time.sleep(0.5)
```

```
[张量点乘测试] 开始
[[张量点乘测试]] 耗时 0.5053489208221436 秒
```

try 语句学习

try 语句尝试

[18]:
```python
data = [10.5, 20.1, "N/A", 30.2, 0, 15.0]
y=[]
for i in data:
    try:
        y.append(100/i)
    except TypeError as e:print(f"[数据类型错误] 无法计算 x={i}, 错误信息: {e}")
    except ZeroDivisionError as e:print(f"[数学奇点] 发现除零操作 x={i}, 错误信息: {e}")
    except Exception as e:
```

```
        print(f"[Warning] 发生了{e} ,x={i}")
    else:
        print("[Success] No Warnings happened.")
    finally:
        print(f"end")
print(f"\nfinal result:{y}")
```

```
[Success] No Warnings happened.
end
[Success] No Warnings happened.
end
[数据类型错误] 无法计算 x=N/A, 错误信息: unsupported operand type(s) for /: 'int'␣
↪and 'str'
end
[Success] No Warnings happened.
end
[数学奇点] 发现除零操作 x=0, 错误信息: division by zero
end
[Success] No Warnings happened.
end

final result:[9.523809523809524, 4.975124378109452, 3.3112582781456954,
6.666666666666667]
```

try 语句实践

[19]:
```
G = 1
class SpaceTimeAnomalyError(Exception):pass
def calculate_gravity(m1,m2,r):
    global G
    if r<=0:
        raise SpaceTimeAnomalyError
    return G*m1*m2/r**2
try:
    F = calculate_gravity(2,3,0)
except SpaceTimeAnomalyError as e:
    print("[Warning] 距离为非正数")
```

```
[Warning] 距离为非正数
```

try 的终极挑战　任务背景

假设你正在为射电望远镜编写一个数据预处理脚本。望远镜传回了一组双星系统的数据，包含主星质量 m1、伴星质量 m2 和它们之间的距离 r。由于宇宙射线干扰和传感器老化，数据里充满了各种“噪声”和“物理奇点”。###### 挑战要求请编写一个完整的脚本 astro_pipeline.py，要求如下：* 自定义异常：定义一个 NegativeMassError（负质量错误），如果质量 m_1 或 m_2 小于 0，必须主动 raise 这个异常（因为在经典物理中，质量不能为负）。* 万有引力函数：编写 calculate_gravity(m1, m2, r)，包含上面提到的负质量检查，以及当 r 0 时抛出内置的 ValueError。* 数据处理主循环：给定以下“脏数据”列表，使用 for 循环遍历它：star_data = [{“m1”: 10, “m2”: 20, “r”: 5}, # 正常数据 {“m1”: 15, “m2”: “Unknown”, “r”: 8}, # 传感器乱码 (类型错误) {“m1”: 30, “m2”: 40, “r”: 0}, # 黑洞视界交叠 (除零错误 / 值错误) {“m1”: -5, “m2”: 25, “r”: 10}, # 反物质/暗能量干扰 (负质量错误) {“m1”: 50, “m2”: 50, “r”: 10} # 正常数据]

- 异常防御网：在循环内部，使用一个完整的 try…except…else…finally 结构：
  - 捕获 TypeError（并打印“数据损坏”）。
  - 捕获 ValueError 或 ZeroDivisionError（并打印“空间坐标重合”）。
  - 捕获你自定义的 NegativeMassError（并打印“检测到物理法则异常”）。
  - 在 else 中打印并保存计算成功的引力结果。
  - 在 finally 中打印“当前星系数据扫描结束”。

代码如下

[20]:
```
class NegativeMassError(Exception):pass
G=1
def calculate_gravity(m1,m2,r):
    global G
    if min(m1,m2)<0:
        raise NegativeMassError
    if r<=0:
        raise ValueError("距离不能非正")
    return G*m1*m2/r**2
def main():
    star_data = [
    {"m1": 10, "m2": 20, "r": 5},       # 正常数据
    {"m1": 15, "m2": "Unknown", "r": 8}, # 传感器乱码（类型错误）
    {"m1": 30, "m2": 40, "r": 0},       # 黑洞视界交叠（除零错误 / 值错误）
    {"m1": -5, "m2": 25, "r": 10},      # 反物质/暗能量干扰（负质量错误）
    {"m1": 50, "m2": 50, "r": 10}       # 正常数据
```

```
    ]
    result = []
    for data in star_data:
        try:
            y=calculate_gravity(data['m1'],data['m2'],data['r'])
        except TypeError:
            result.append("数据损坏")
        except (ValueError,ZeroDivisionError):
            result.append("空间坐标重合")
        except NegativeMassError:
            result.append("检测到物理法则异常")
        else:
            result.append(y)
        finally:
            print("当前星系数据扫描结束")
    print(f"最终结果: {result}")
if __name__ == '__main__':
    main()
```

```
当前星系数据扫描结束
当前星系数据扫描结束
当前星系数据扫描结束
当前星系数据扫描结束
当前星系数据扫描结束
最终结果: [8.0, '数据损坏', '空间坐标重合', '检测到物理法则异常', 25.0]
```

with 的综合挑战

期末大挑战：构建“容错型物理仿真日志系统” 至此，你已经完全掌握了 with 语句的底层魔法（**enter/exit**）和高阶写法（@contextmanager）。

- 背景：你正在运行一个包含多个独立批次的流体力学仿真脚本。偶尔，某个批次会因为极端的边界条件触发 ZeroDivisionError（除以零，我们可以容忍并跳过），但如果触发了 ValueError（比如矩阵发散），我们就必须立刻停止整个实验。
- 任务：请你用类（Class）的方式，编写一个名为 SimulationLogger 的上下文管理器。（在这个场景下，因为需要根据不同的错误类型决定是否拦截，使用类和 **exit** 的三个参数会让逻辑更清晰）。
- 具体要求：

1. 接收一个参数 filename（日志文件名）。
2. 在进入时（**enter**），打开这个文件准备追加写入（模式为 “a”），并向文件中写入一行：“— 仿真批次开始 —”，同时把文件对象传递给 as。
3. 如果 with 块内部运行正常，退出时（**exit**）向文件写入：“仿真成功结束”。
4. 核心容错逻辑：如果在退出时发现发生了 ZeroDivisionError，请向日志写入 “[警告] 遇到除以零错误，已忽略”，并拦截这个错误（让程序活下来）；如果发生其他错误，向日志写入 “[致命] 发生严重错误，系统崩溃”，并放任其崩溃。
5. 无论发生什么，必须在最后安全地关闭文件。

测试代码模板

```
# 请在这里实现你的 SimulationLogger 类
# ...

print(" 启动全天候仿真管道...")

# 批次 1: 正常运行
with SimulationLogger("physics_sim.log") as log_file:
    log_file.write(" 批次 1: 计算流场...\n")

# 批次 2: 发生可容忍的错误
with SimulationLogger("physics_sim.log") as log_file:
    log_file.write(" 批次 2: 尝试激波计算...\n")
    1 / 0  # 触发除以零，应该被拦截

# 批次 3: 发生致命错误
with SimulationLogger("physics_sim.log") as log_file:
    log_file.write(" 批次 3: 网格生成...\n")
    raise ValueError(" 网格出现负体积，矩阵发散！")

print(" 这行代码永远不应该被打印，因为批次 3 应该导致程序崩溃。")
```

我的代码

[21]:
```
class SimulationLogger:
    def __init__(self,filename):
        self.filename = filename
    def __enter__(self):
        self.file = open(self.filename,"a")
```

```
        self.file.write("仿真批次开始")
        return self.file
    def __exit__(self,exc_type,exc_val,tb):
        if exc_type is ZeroDivisionError:
            self.file.write("[警告] 遇到除以零错误，已忽略")
            self.file.close()
            return True
        elif exc_type is not None:
            self.file.write("[致命] 发生严重错误，系统崩溃")
        else:
            self.file.write("仿真成功结束")
        self.file.close()
```

[22]:
```
# 请在这里实现你的 SimulationLogger 类
# ...

print("启动全天候仿真管道...")

# 批次 1: 正常运行
with SimulationLogger("physics_sim.log") as log_file:
    log_file.write("批次 1: 计算流场...\n")

# 批次 2: 发生可容忍的错误
with SimulationLogger("physics_sim.log") as log_file:
    log_file.write("批次 2: 尝试激波计算...\n")
    1 / 0  # 触发除以零，应该被拦截

# 批次 3: 发生致命错误
with SimulationLogger("physics_sim.log") as log_file:
    log_file.write("批次 3: 网格生成...\n")
    raise ValueError("网格出现负体积，矩阵发散！")

print("这行代码永远不应该被打印，因为批次 3 应该导致程序崩溃。")
```

启动全天候仿真管道…

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[22], line 18
     16 with SimulationLogger("physics_sim.log") as log_file:
     17     log_file.write("批次 3: 网格生成…\n")
---> 18     raise ValueError("网格出现负体积，矩阵发散！")
     20 print("这行代码永远不应该被打印，因为批次 3 应该导致程序崩溃。")

ValueError: 网格出现负体积，矩阵发散！
```

[ ]:
```python
with open("physics_sim.log", "r") as f:
    content = f.read()
    print(content)
```

```
仿真批次开始批次 1: 计算流场…
仿真成功结束仿真批次开始批次 2: 尝试激波计算…
[警告] 遇到除以零错误，已忽略仿真批次开始批次 3: 网格生成…
[致命] 发生严重错误，系统崩溃仿真批次开始批次 1: 计算流场…
仿真成功结束仿真批次开始批次 2: 尝试激波计算…
[警告] 遇到除以零错误，已忽略仿真批次开始批次 3: 网格生成…
[致命] 发生严重错误，系统崩溃仿真批次开始仿真批次开始批次 1: 计算流场…
仿真成功结束仿真批次开始批次 2: 尝试激波计算…
[警告] 遇到除以零错误，已忽略仿真批次开始批次 3: 网格生成…
[致命] 发生严重错误，系统崩溃
```

1.1.4 参数更新过程

此过程中必须把自动求导的计算图关掉

[24]:
```python
learning_rate = 0.001
with torch.no_grad():
    W -= learning_rate * W.grad
    b -= learning_rate * b.grad
    W.grad.zero_()
    b.grad.zero_()
```

解释 在 PyTorch 中，只要一个张量设置了 requires_grad=True，它身上就带了两个东西：* 监控录像机：只要你对这个张量做数学运算（加减乘除），PyTorch 就会在后台画一张“计算图”（Computational Graph），记录下操作的轨迹。* 垃圾桶（梯度缓存）：也就是 W.grad 这个属性。当你调用 loss.backward()（时间反演）时，录像机里的记录会被消耗掉，算出来的梯度数值会被统统扔进这个“垃圾桶”里。* W.grad.zero_() 做了什么？它非常忠诚地把垃圾桶里的数值变成了 0。它绝对不会清空或影响计算图。* 当你写下 with torch.no_grad(): 时，你就是在告诉 PyTorch：“嘿，我现在要以上帝视角手动把这个粒子 W 挪动一下位置。请把监控录像机关掉，不要把我的挪动记录到你的计算图里。”

1.2 torch.nn 学习

1.2.1 番外：使用 nn.Module 自建一个非线性层

[25]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F

class MyNonLinearLayer(nn.Module):
    def __init__(self, alpha_init=1.0, beta_init=1.0):
        super().__init__()
        # 必须包装成 nn.Parameter，否则优化器找不到它
        self.alpha = nn.Parameter(torch.tensor([alpha_init]))
        self.beta = nn.Parameter(torch.tensor([beta_init]))

    def forward(self, x):
        # 使用 PyTorch 的张量运算，保证自动求导链条不断裂
        # y = alpha * ln(1 + exp(beta * x))
        return self.alpha * torch.log(1 + torch.exp(self.beta * x))

# 测试
layer = MyNonLinearLayer(alpha_init=0.5)
input_data = torch.randn(5)
output = layer(input_data)

print(f"输出: {output}")
print(f"可学习参数: {list(layer.parameters())}")
```

```
输出: tensor([0.3883, 0.4474, 0.1746, 0.8177, 0.8343], grad_fn=<MulBackward0>)
可学习参数: [Parameter containing:
tensor([0.5000], requires_grad=True), Parameter containing:
tensor([1.], requires_grad=True)]
```

1.2.2 用 torch 的零件组装网络

[26]:
```
import torch.nn as nn
my_net = nn.Sequential(nn.Linear(2,50),
                       nn.ReLU(),
                       nn.Linear(50,10),
                       nn.ReLU(),
                       nn.Linear(10,2),
                       )
my_loss = nn.MSELoss()
```

1.2.3 优化更新 W 和 b

[27]:
```
my_optimizer = torch.optim.SGD(my_net.parameters(), lr=0.001)
```

1.2.4 组合起来变成训练循环

[28]:
```
for i in range(10000):
    my_optimizer.zero_grad()
    y_pred = my_net(x) # 相当于 y = my_net.forward(x)
    loss = my_loss(y_pred,y_true) # 相当于 loss = my_loss.forward(y,y_true)
    loss.backward()
    my_optimizer.step()
```

[29]:
```
print(loss)
```

```
tensor(0.0003, grad_fn=<MseLossBackward0>)
```

1.3 torch.autograd 学习

1.3.1 叶子节点与非叶子节点

- 叶子节点 (Leaf Node):
  - 由用户直接分配创建的张量，它们不是任何计算的结果（通常是你的模型参数、初始物理状态等）。
  - 求 `leaf.grad`：可以令 `leaf.requires_grad = True`

- 非叶子节点 (Non-leaf Node):
  - 通过张量数学运算产生的中间张量（比如激活函数的输出）。
  - 求 `nonleaf.grad` ：不可以令 `nonleaf.requires_grad = True` ，只能用 `nonleaf.retain_grad()` 告诉 PyTorch：手下留情，别把 `nonleaf` 的梯度删了
- 边 (Edges) 与操作 (grad_fn):
  - 图中的边代表了具体的数学函数，每个非叶子节点都有一个 grad_fn 属性，它不仅知道前向是怎么算的，更内置了该步骤的局部偏导数公式。

[30]:
```
x = torch.tensor([2.0], requires_grad=True)
y = x ** 2
y.retain_grad() # 告诉 PyTorch: 手下留情，别把 y 的梯度删了！
z = 3 * y
z.backward()
print(x.grad)
print(y.grad) # 这次就能打印出结果了！
```

```
tensor([12.])
tensor([3.])
```

1.3.2 张量对张量求导

因为 y 不是标量，所以直接 `y.backward()` 会报错

应该使用 `y.backward(gradient=v)` ，`x.grad` 会输出一个向量 $\vec{v}^T \frac{\partial \vec{y}}{\partial \vec{x}} = \frac{\partial(\vec{v}^T \vec{y})}{\partial \vec{x}}$

[31]:
```
x = torch.tensor([1.0, 2.0], requires_grad=True) # x1=1.0, x2=2.0
y = torch.empty(2)
y[0] = x[0] ** 2 + 3 * x[1]  # y1 = x1^2 + 3*x2
y[1] = x[1] ** 3 + 2 * x[0]  # y2 = x2^3 + 2*x1
v = torch.tensor([1,0])
y.backward(gradient=v)
print(x.grad)
```

```
tensor([2., 3.])
```

1.3.3 `.detach()` 的用法

- 当你在一个张量上调用 y = x.detach() 时，发生了两件事：
  1. **内存共享（深层绑定）**：y 和 x 指向物理内存中的同一块数据。如果你修改了 y 的值，x 的值也会变（所以还是要小心就地操作！）。

2. **因果断裂（图的截断）**：y 是一个全新的张量，它的 `requires_grad` 被强制设为了 `False`。更重要的是，y 没有任何 `grad_fn`，它变成了一个新的叶子节点。PyTorch 的反向传播引擎在往回追溯时，一旦碰到 y，就会认为 "线索到此为止"，绝不会继续往上追溯到 x。

- 极其常见的实用场景：
  - 在 PyTorch 中，如果你想把一个带有梯度的张量转换成 NumPy 数组以便用 Matplotlib 画图，直接调用 x.numpy() 会报错。你必须先剥离它的梯度图：`x_numpy = x.detach().numpy()`
  - 把不需要求导的部分用`.detach()` 取消计算图的生成，减小内存占用：`L_physics = (p.detach() ** 3)`

1.3.4 求高阶导数

全导数

[32]:
```
t = torch.tensor([2.0], requires_grad=True)
s = t ** 3

# 计算一阶导：速度 v (ds/dt)
# 注意：grad() 返回的是一个元组，即使只有一个输入，所以我们需要加上 [0] 提取张量
v = torch.autograd.grad(
    outputs=s,
    inputs=t,
    create_graph=True  # 核心：为计算速度的过程建图！
)[0]

print(f"速度 v = {v.item()}") # 输出 12.0

# 计算二阶导：加速度 a (dv/dt)
# 这一次我们不需要 create_graph=True，因为我们只求到二阶，不需要继续往下求三阶了
a = torch.autograd.grad(
    outputs=v,
    inputs=t
)[0]

print(f"加速度 a = {a.item()}") # 输出 12.0
```

```
速度 v = 12.0
加速度 a = 12.0
```

偏导数

[33]:
```python
V = torch.tensor([2.0], requires_grad=True)
T = torch.tensor([300.0], requires_grad=True)

# 计算压强 P
P = T / V

# 同时对 V 和 T 求偏导
# 注意看 inputs 接收了一个元组 (V, T)
grads = torch.autograd.grad(outputs=P, inputs=(V, T))

# 返回的 grads 就是一个包含两个元素的元组
dP_dV = grads[0]  # 对应 inputs 里的第一个元素 V
dP_dT = grads[1]  # 对应 inputs 里的第二个元素 T

print(f"dP/dV: {dP_dV.item()}") # 物理意义: 等温压缩率相关
print(f"dP/dT: {dP_dT.item()}") # 物理意义: 等容膨胀系数相关
```

```
dP/dV: -75.0
dP/dT: 0.5
```

张量对张量求导

[34]:
```python
t = torch.tensor([2.0,3.0], requires_grad=True)
s = t ** 3
e1 = torch.tensor([1,0])

# 计算一阶导: 速度 v (ds/dt)
# 注意: grad() 返回的是一个元组, 即使只有一个输入, 所以我们需要加上 [0] 提取张量
v1 = torch.autograd.grad(
    outputs=s,
    inputs=t,
    create_graph=True,
    grad_outputs=e1 # 点乘 e1, 由雅克比矩阵变为向量
)[0]

print(f"速度 v = {v1.detach().cpu().numpy()}") # 输出 12.0

# 计算二阶导: 加速度 a (dv/dt)
```

```python
# 这一次我们不需要 create_graph=True，因为我们只求到二阶，不需要继续往下求三阶了
a1 = torch.autograd.grad(
    outputs=v1,
    inputs=t,
    grad_outputs=e1 # 点乘 e1，由雅克比矩阵变为向量
)[0]

print(f"加速度 a = {a1.detach().cpu().numpy()}") # 输出 12.0
```

```
速度 v = [12.  0.]
加速度 a = [12.  0.]
```